

Construindo a Web API

Fundacional da Mecânica

Autores:

Emanuel de Souza Alves
Gustavo Steven Lima Sousa
Leonardo Rocha de Lima Silva
Guilherme Viana Oliveira
Yuri Tigre de Oliveira de Souza
Felipe Natan da Silva

Agenda

Evolução da OAT 1: novas entidades, padrões de projeto e documentação da API.

1. Contexto

Base da OAT 1 e escopo atual

2. Modelagem

OS, pedidos e ItemPedido

3. Padrões

Singleton, Builder e DTOs

4. Diagramas

Classes e atividade do Controller

5. Conclusão

Resultados e ajustes pendentes

Contexto do projeto

- MecâniQA: gestão de autopeças, auto centers e oficinas.
- OAT 1: CRUD de Peça e Serviço em Java com Spring Boot.
- OAT 2: ordens de serviço, pedidos de peças e seus status.
- Persistência em memória com Singleton, sem banco de dados.

Novas entidades

- OrdemServico reúne cliente, veículo, serviços e itens de peças.
- PedidoPecas reúne data, status e uma lista de ItemPedido.
- ItemPedido associa a peça ao pedido e registra quantidade, preço unitário e subtotal.

Status da OS: Aberto, Pendente de Pagamento, Pago, Em Execução e Executado.

Status do pedido: Orçando, Pendente de Pagamento, Pago/Faturado e Entregue.

Singleton e Builder

Singleton

Uma instância compartilhada do repository.

```
private static OrdemServicoRepository
    instance;

private OrdemServicoRepository() {
    this.ordensServico = new ArrayList<>();
    this.proximoId = 1L;
}

public static synchronized
OrdemServicoRepository getInstance() {
    if (instance == null) {
        instance = new OrdemServicoRepository();
    }
    return instance;
}
```

Builder da OS

Construção da OS por métodos encadeados.

```
public OrdemServicoBuilder
comCliente(String cliente) {
    this.cliente = cliente;
    return this;
}

// Uso em OrdemServicoMapper:
return new OrdemServicoBuilder()
    .comCliente(dto.getCliente())
    .comVeiculo(dto.getVeiculo())
    .comStatus(dto.getStatus())
    .comServicos(servicos)
    .comItensPeca(itensPeca)
    .build();
```

DTOs e Mappers

- RequestDTO recebe os dados da requisição. ResponseDTO define os dados devolvidos.
- OrdemServicoMapper converte o DTO usando o Builder e prepara a resposta com o valor total.
- PedidoPecaMapper converte pedidos e seus itens entre DTOs e entidades.

Diagrama de classes

Entidade associativa

Um pedido contém vários itens. Cada ItemPedido referencia uma peça e armazena sua quantidade e preço.

Subtotal = quantidade × preço unitário.
O total do pedido soma os subtotais.

Ajuste no diagrama completo: representar OrdemServicoBuilder como classe separada e alinhar os DTOs ao código.

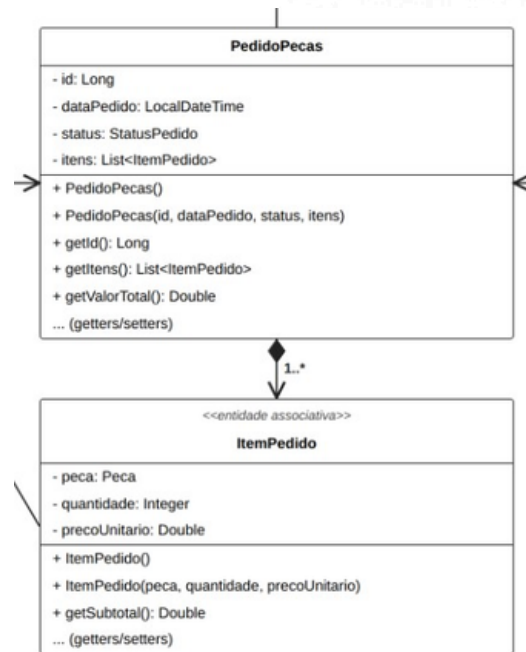
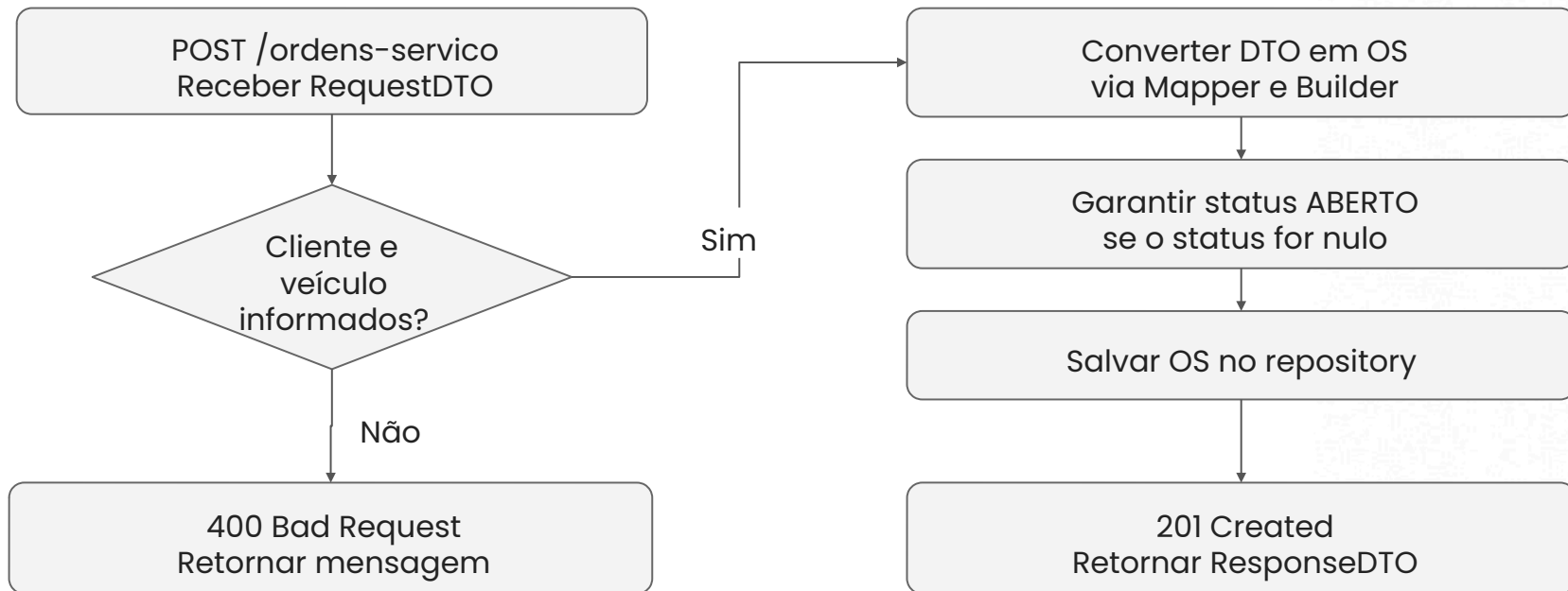


Diagrama de atividade



Fluxo do Controller conforme o código atual. O Mapper chama o Builder; IDs inexistentes não geram retorno 404 na criação.

Considerações finais

- A API amplia o CRUD inicial com OS, pedidos, inclusão de itens e atualização de status.
- Builder e DTOs/Mappers estão presentes nos fluxos de OS e pedidos.
- Ajustes: DTOs em Peça e Serviço e alinhamento dos diagramas com a implementação.
- O repositório contém uma collection Postman para apoiar a demonstração dos endpoints.

Referências



- Documento da OAT 2: Padrão de Projeto e Diagrama de Atividade.
- Apresentação da OAT 1 e diagramas fornecidos pela equipe Macapá.
- Código da equipe: Emanuel-bit631/mecaniQA-api-equipe-macapa

<https://github.com/Emanuel-bit631/mecaniQA-api-equipe-macapa>

Consulta à branch main em 28/09/2026. Commit dd3d91d.